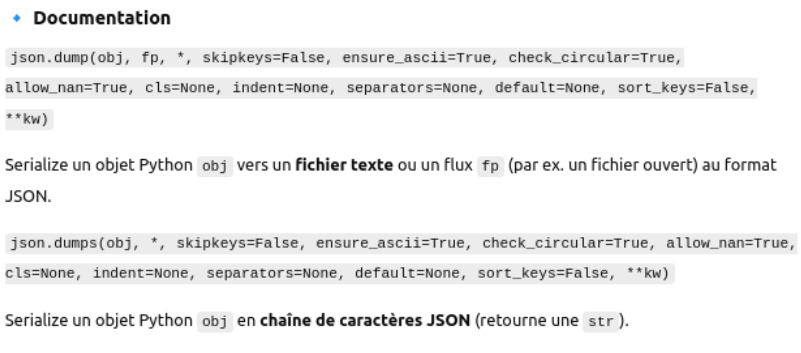
Techniques web : Contrôle 1

Nom
Prénom
N étudiant
Université

I) Questions de cours (/12, chaque question /2)

1) Dumping sauvage

```
import json
data = {"nom": "Alice", "age": 30,
"langages": ["Python", "C++"]}
json_texte = json.dumps(data)
with open("data.json", "w") as f:
    json.dump(data, f)
with open("data.json", "r") as f:
    contenu = f.read()
```



Documentation

`json.dump(obj, fp, *, skipkeys=False, ensure_ascii=True, check_circular=True, allow_nan=True, cls=None, indent=None, separators=None, default=None, sort_keys=False, **kw)`

Serialize un objet Python `obj` vers un **fichier texte** ou un flux `fp` (par ex. un fichier ouvert) au format JSON.

`json.dumps(obj, *, skipkeys=False, ensure_ascii=True, check_circular=True, allow_nan=True, cls=None, indent=None, separators=None, default=None, sort_keys=False, **kw)`

Serialize un objet Python `obj` en **chaîne de caractères JSON** (retourne une `str`).

À quoi servent les fonctions `dump()` et `dumps()` du module `json` ?
Quelle est la différence entre les deux ?
Dans quel cas utiliser l'une ou l'autre ?
Expliquez à partir du code donné en exemple ci-dessus et de la documentation.

2) Un tableau pour les définir tous

Complétez le tableau suivant en précisant pour chaque langage :

S'il agit côté client ou côté serveur, son rôle principal dans le développement Web,un ou deux exemples concrets d'usage

Langage	Côté (client / serveur)	Rôle principal	Exemples d'usage	Ordre d'apparition
JavaScript				
HTML				
Node.js				
CSS				

3)Vous êtes le panda :

🐼 Exercice : You are the pandas

On dispose du tableau suivant sur des pandas dans un zoo :

Nom	Age	Sexe	Poids_kg	Zone
Pandy	5	M	80	Nord
Lulu	2	F	40	Sud
Bao	7	M	90	Est
Mei	4	F	50	Ouest
Tao	3	M	60	Sud

Ecrivez le résultat des lignes de code suivantes :

print(pandas_zoo.head(2))

print(pandas_zoo['Age'].mean())

print(pandas_zoo['Sexe'].value_counts())

pandas_zoo['Categorie_Poids'] = pandas_zoo['Poids_kg'].apply(lambda x: 'Léger' if x < 60 else 'Lourd')
print(pandas_zoo)

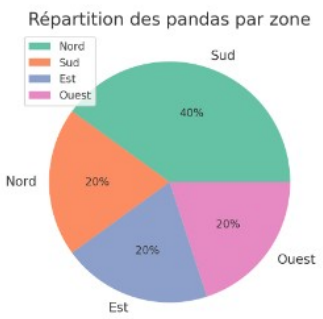
4) Encore plus de pandas

Le code ci-dessous crée deux graphiques à partir d'un DataFrame Pandas. Expliquez ligne par ligne ce qu'il réalise et à quoi servent les légendes.

```
plt.figure(figsize=(6,4))
plt.bar(df["Nom"], df["Poids_kg"], color="lightgreen")
plt.title("Poids des pandas 🐼")
plt.xlabel("Nom")
plt.ylabel("Poids (kg)")
plt.legend(["Poids individuel"])
plt.show()

plt.figure(figsize=(5,5))
df["Zone"].value_counts().plot(kind="pie", autopct="%1.0f%%", c
plt.title("Répartition des pandas par zone")
plt.ylabel("")
plt.legend(df["Zone"].unique(), loc="upper left")
plt.show()
```





5) Rapide et efficace : Réécrivez les 4 extraits de code suivants de façon idiomatique

```python
nombres = [1, 2, 3, 4, 5, 6]
pairs = []
for n in nombres:
    if n % 2 == 0:
        pairs.append(n)
print(pairs)
```

```python
cles = ["nom", "âge", "ville"]
valeurs = ["Alice", 30, "Paris"]
personne = {}
for i in range(len(cles)):
    personne[cles[i]] = valeurs[i]
print(personne)
```

```python
x = 10
y = 20
tmp = x
x = y
y = tmp
print(x, y)
```

```python
mots = ["Python", "code", "élégant", "lisible"]
tous_longs = True
for mot in mots:
    if len(mot) < 5:
        tous_longs = False
print(tous_longs)
```

6) Les précautions :

```
import json
import os

dossier = "donnees_json"

for nom_fichier in os.listdir(dossier):
    if nom_fichier.endswith(".json"):
        chemin = os.path.join(dossier, nom_fichier)

        with open(chemin, "r", encoding="utf-8") as f:
            data = json.load(f)

        data["universite"] = "INALCO"

        with open(chemin, "w", encoding="utf-8") as f:
            json.dump(data, f, ensure_ascii=False, indent=4)
```

Que fait ce code ? Qu'est ce qui pourrait mal se passer ?Citez au moins deux sources d'erreur possible.
Réécrivez ce code avec toutes les bonnes pratiques pour s'assurer qu'il s'exécute bien.

II) Algorithmique (/10)

Voici un code en python

```python
def encore_des_pandas(num):
    pandas_string = []
    sign_to_unit = {"V": "I", "X": "I", "L": "X", "C": "X", "D": "C", "M": "C"}
    value_to_sign = {1000: 'M', 500: 'D', 100: 'C', 50: 'L', 10: 'X', 5: 'V', 1: 'I'}

    for step in sorted(value_to_sign.keys(), reverse=True):
        roman_letter = value_to_sign[step]
        division = num // step
        if division > 0:
            pandas_string.extend([roman_letter] * division)
            num -= division * step

        reste_dizaine_suivante = int(str(num % step)[0]) if len(str(num)) == len(str(step)) - 1 else 0

        if reste_dizaine_suivante == 4 and int(str(step)[0]) != 5:
            num += step / 10
            pandas_string.append(sign_to_unit[roman_letter])

        if reste_dizaine_suivante == 9 and int(str(step)[0]) != 5:
            num -= 9 * step / 10
            pandas_string.append(sign_to_unit[roman_letter])
            pandas_string.append(roman_letter)
    return pandas_string
```

1) Expliquez à quoi il sert.
Exécutez l'algorithme avec en entrée 419 et profitez-en pour expliquer le code ligne à ligne. /2

2) Quelle est la complexité de ce programme ? Justifiez /1

3) Que pourrait-on écrire pour écrire les lignes 6 et 7 en une seule ligne ? /1

4) Pourquoi doit-on traiter le cas du 4 et du 9 différemment ? /1

5) Cet algorithme « lit le chiffre de gauche à droite ». /5
Écrivez un algorithme qui fait la même chose mais en lisant la chaîne dans l'autre sens (c'est à dire en commençant par les unités).